Our Team Git Workflow Guide

Using the 'test' Branch as Our Shared Base (Instead of main)

> ⭐ **THE GOLDEN RULE**
>
> **Forget that the `main` branch exists for now.** Our absolute newest, most updated code lives on the `test` branch. Never code directly inside `test`. Always build on your own branch, then copy it into `test` when it is fully working.

1. Sitting Down to Work (Getting Ready)

Run these commands first before touching any code files so you do not build on top of outdated versions:

Command to Type	When & Why to Use It
`git checkout test`	Use this to safely stand on the team's shared tracking branch before doing anything else.
`git pull origin test`	Use this immediately after switching to test. This downloads your team's latest code updates into your laptop.
`git checkout -b feature/your-name`	Use this right after pulling. It creates your own private branch (your sandbox) and places you inside it. **You are now safe to code.**

2. Writing Code (Saving Game Progress)

Run these commands every hour or whenever you finish a piece of work (like a screen layout, a route, or an asset fix) to create local checkpoints:

Command to Type	When & Why to Use It
`git add .`	Use this when your new code works and you want Git to track your modifications.
`git restore --staged <file>`	Use this if you accidentally staged a file (like local server testing files) that you do not want to share with the group.
`git commit -m "added a message"`	Use this right after `git add .` to officially lock your changes into your laptop's local history ledger.

3. Sharing and Combining Code (The Hand-off)

When your assignment is completed and you are ready to combine it with the main `test` deployment branch, follow this exact recipe:

1. Upload your private branch safely to GitHub so your team can access it:

```
git push origin feature/your-name
```

2. Switch your computer back over to the team's tracking headquarters branch:

```
git checkout test
```

3. Pull down any changes someone else might have submitted while you were coding:

```
git pull origin test
```

4. Mix your working feature branch directly into the test branch:

```
git merge feature/your-name
```

Note: If a dark empty terminal screen opens up here, press `Esc`, type `:wq`, and hit `Enter` to save and close it.

5. Push the newly combined code up to GitHub so everyone else stays in sync:

```
git push origin test
```

4. Panic Mode (The Emergency Brakes)

If a command causes your sidebar to turn red with exclamation marks (`!`) or conflict indicators, use these options to reset safely:

> 🚨 **Emergency Command: git merge --abort**
>
> Use this if a merge or pull triggers code conflicts and you feel overwhelmed. Running this instantly cancels the entire command and rolls your project back to exactly how it was 5 seconds before you hit Enter.

> 🚨 **Extreme Clean Slate Command: git reset --hard HEAD**
>
> Use this if your local code is broken, nothing is running, and you want to completely erase your recent local unsaved edits to match your last successful save checkpoint exactly.